

Governed Multi-Agent Architecture

Version 1.6 · 2026-08-11 · Domain-agnostic — instantiate per project Licensed CC BY 4.0 — attribution required.
Canonical home: gmaa.ai · github.com/gmaa-ai/gmaa · cite by version.

A reference architecture for running multiple AI agent seats and multiple human operators against a single codebase, such that no change lands without authorization at the correct unit, no claim closes without evidence, and no failure degrades silently.

Grounding principle (the thesis this architecture implements): **multi-agent systems fail by authorizing at the wrong unit** — individual changes get approved while the coherence of the complete pending set goes unexamined. The correction: an independent human ratifies the *complete set* against system invariants before any member commits. Everything below is machinery in service of that one correction.

1 · Core model

1.0 Definitions (the identity layer, made explicit)

Five words do the identity work of this document; conflating them is where governance failures begin, so they are defined here and used strictly:

- **Seat** — the office: a governance identity with a named role, a charter, an authority class, a territory, and a boundary stateable in one sentence. A seat lives in the record (roster, manifest, charter file) and persists whether or not anything is running. A named-dormant seat is a real seat with an empty chair.
- **Session** — the shift: one context window occupying one seat (§2, ONE-SESSION-ONE-SEAT). Sessions are ephemeral by design — restart-cheap, killed and respawned freely — because the seat's identity and memory live in the record, never in the session.
- **Agent** — the occupant: whatever intelligence currently fills a seat, on any surface — a code session running an agent definition, or a chat session. The agent is the **census unit**: you count agents, you include yourself, and a provisioned-but-empty seat is on the roster but not in the census.
- **Agent definition** — the qualification: the persona/contract a code session loads to become a fit occupant of its seat (knowing to derive identity from the stamp, emit its boot receipt, read its charter and never author it). A session without its definition loaded is an unqualified occupant in a real seat — the seat is fine, the chair is filled wrong, and the definition cannot guard its own absence. (*Instantiation vocabulary: the origin program names its reference definition "orc"; adopters name their own. The canonical term is agent definition.*)
- **Lane** — fleet vocabulary for a seat bound to its worktree territory: seat-plus-territory, not a distinct concept. The relations carry law: **the seat persists, the session occupies, the definition qualifies the occupant**. Seat creation (§7) creates the office before any shift exists; the double-launch guard enforces one shift per office; retirement closes the office, never merely the shift.

1.1 Seats, roles, and the single-writer spine

Work is performed by **seats**. A seat is an agent session (or human surface) bound to:

- a **worktree** (its filesystem territory),
- a **role archetype** (its authority contract),
- an **operator** (the human accountable for it),
- an **allow-list profile** (its pre-authorized operations),
- a **key** (its transport identity — see §6).

Role archetypes are a small fixed set. Seats are instances; projects scale by adding instances, never by inventing authority.

Archetype	Authority	Hard prohibitions
Foundation (writer)	Sole committer to the spine. Executes within ratified authority. Runs reconciliation diffs.	Never self-ratifies scope expansion.
Executor	Runs side-effectful operations (SQL, webhooks, API calls) within its allow-list.	Never commits to the spine. Never widens its own credentials.
Lane	Produces domain output inside one module worktree.	Never commits to the spine. Never carries rulings.
Auditor	Post-commit verification, standing probes, register enforcement.	Never fixes what it finds — it files.
Product Architect (human-adjacent, chat surface)	Authors contracts, specs, relay files, rulings. Ratifies HOW-class decisions. Reviews at framing level, not just fact level.	Never commits. Never rubber-stamps agent output.
Designer (human-adjacent)	Authors view-layer artifacts against a typed seam contract.	Never touches data-layer implementations.
Operator	Budget, set-level ratification, final publish/ship decisions, physical custody (launches, ferries), out-of-band facts, attestations.	Never used as a decision-outsourcing channel for HOW-class questions — those belong to the Architect.

Single-writer invariant: exactly one seat commits to the spine (the canonical branch/repo). All other output reaches the spine *through* that seat. This is what makes distribution trivial later (§8): adding people never adds committers, so there is no merge governance to invent.

Seat surfaces: code seats and chat seats. Seats exist on two surfaces, and the architecture governs both. A **code seat** has a filesystem, a git identity, and a runtime session (§7); it can probe, generate, and — if it is the writer — commit. A **chat seat** (a conversation-interface session, e.g., a project chat) has **none of these**: no filesystem, no git, no CLI. Chat-seat law, identical in every program:

- A chat seat **authors and never commits**. Its artifacts become canon only when a code seat or the operator commits them.

- Every repo path a chat seat cites names a location on someone else's disk. Files uploaded to a chat seat are **ferried snapshots** — current when carried, stale afterward; version and HEAD truth come from receipts or the operator's word, never from assuming an uploaded copy is live.
- The **operator ferry** — a human carrying files between chat seats and the filesystem — is a sanctioned transport with spool semantics (§5): the ferried file is the message, delivery is placement, and substantive content travels only as files on the record. A chat seat never treats conversational paste as landed state.
- A chat seat has no live view of any other seat. What another seat "did" is known only through ferried receipts — no receipts, no claim.

Role assessment (chat and Code surfaces both). An architect-surface session's first boot act, before assuming any authority, is role assessment: determine from the project's charter and context which architect role the session serves, and assume exactly that role. The roles derive from the custody split (three layers, two homes): **Product Architect** — custody of a portable layer (the architecture/claims/evidence corpus of a product or program: document, amendment queue, channel); **Solution Architect** — custody of an instantiation layer (conforming one project or solution to a canonical it does not own). The assessment result is stated in the boot report; a context supporting neither reading is a halt-loud, never a default to Product Architect. A mis-assessed role is a wrong-authorization-unit failure at the identity layer.

1.2 The two-plane authorization model

Every operation is authorized on two independent planes:

- **Governance plane** — rulings, ratifications, contracts. Answers: *is this class of action authorized at all, and at what unit?*
- **Harness plane** — the agent runtime's permission system (modes, allow-lists, prompts). Answers: *may this specific invocation proceed right now?*

The planes must agree by construction, not by vigilance. Rule: **capability follows authorization** — no credential grant, allow-list entry, or permission mode may contain operations the governance plane has not ratified. The harness is the enforcement surface of the governance plane, never a second source of authority.

1.3 Subagents — work fans out, authority does not

Seats may spawn **subagents**: ephemeral workers running inside the seat's own session to parallelize work. Subagents are not seats. The distinction is load-bearing:

- **No identity, no authority.** A subagent has no manifest entry, no key, no allow-list of its own, and no seat instruction file. It operates strictly inside its parent seat's ratified authority and permission envelope — a subagent can never do what its parent seat may not, and spawning subagents never widens what the seat may do. Authority never fans out; only work does.
- **The parent owns everything.** Every subagent action, claim, and artifact attributes to the parent seat. A subagent's report is *input to the seat*, not system fact; the seat verifies before it relies. Subagent summaries are memory-not-evidence — results return as artifacts and receipts the parent checks, never as narration the parent forwards.

- **Fan-out rule: parallel validation, serial commit.** A seat may fan subagents across work items only when their scopes are *provably disjoint* — judged on **two surfaces**: the work scope (files/objects touched) **and** the write target (what each result will alter in shared state). Two items can touch disjoint files yet both append to one ledger or alter one table; that is not disjoint. Where disjointness cannot be proven, the default is serial. Whatever the fan-out, the parent unions results and commits **once, serially** — parallelism ends at the boundary.
- **Role restrictions.** Enforcement roles are never discharged by ephemeral subagents: the auditor is committed machinery (a hook, a probe), and the writer's commit act is never delegated downward. A gate that exists only while a subagent is alive is not a gate.

Subagent role classes (bounded roster, ratified definitions). Subagents are not generic helpers; each is an instance of a named role class, and the class determines what tasks it may carry:

Class	Purpose	Hard constraints
Read-only prober / attestor	Verification, attestation, evidence gathering in fan-out	Structurally incapable of mutation; every run is followed by a no-mutation verification, not trusted on instruction alone
Validator	Checks candidate work against contracts and invariants	Returns findings as artifacts; never fixes what it finds; never commits
Builder / worker	Produces work product within a bounded, ratified scope	Own branch/file territory only; output returns to the parent for union; never touches the spine
Advancing executor	Advances a defined multi-step process end-to-end	Sanctioned only for tasks whose intent is advancement; never assigned read-only or verification tasks — an advancing executor given a look-don't-touch task is a structural mismatch regardless of how explicit the instruction is

Class rules, all ruled law in the origin program:

- **Role-task match is mandatory.** The task's nature selects the class; instructions do not convert one class into another. "Read-only" as a prompt on an advancing executor is a wish, not a control.
- **Definitions are committed, versioned artifacts** in the repo (e.g., an agents directory), ratified into a **bounded roster** — never invented ad hoc mid-session.
- **Stored is not loadable.** A ratified definition is not runtime-valid until a live spawn proves the runtime actually loads it. Verify loadability before any plan depends on a subagent; a definition the runtime cannot load is a named wait, not a silent fallback to a different class.
- **Intra-seat only.** Cross-session or multi-session agent-team primitives are prohibited as subagent substitutes. Anything that outlives or crosses a session is a seat — with identity, manifest entry, and governance — never an oversized subagent.
- **Cost envelope (ruled, not assumed).** Subagent fan-out multiplies token cost — order-of-magnitude multipliers are verified in the origin program's receipts, not hypothetical. Each instantiation rules a cost envelope for subagent-heavy flows (model floor and/or batch limits) as an invariant with receipts. Uncontrolled cost is a silent-degradation vector:

it pressures exactly the shortcuts (skipped validation, smoke bypasses, mode escalation) the invariants prohibit. Pricing the boundary's overhead honestly is part of proving it viable.

2 · The invariant pattern

Every project instantiation carries a short list of **binding invariants** in its seat instruction file — load-bearing rules, not aspirations. Four are universal; the rest are domain instantiations declared in slots.

Universal invariants (ship with the template):

1. **LIVE-DATA-ONLY** — no smoke-bypass paths, no hand-inserted test rows posing as production state, no fake data standing in for real substrate. **Declared-substrate ruling:** an adversarial or seeded test substrate with committed ground truth (a seed manifest / `groundtruth.json`) is *declared substrate*, not fake data — the invariant's target is **undeclared** fakes posing as production state. Any data not declared in the seed manifest appearing in a test environment, or any synthetic data appearing in a production environment, is a violation. A seeded trap that slips through detection is a build defect.
2. **LEAST-PRIVILEGE-WRITER** — the workload identity that agents run under is read-only against the substrate by default; privileged operations route through a separately-authenticated human-owned path. (Verified continuously by a standing probe, not asserted.)
3. **NO FAKE CLOSURES** — a task is closed by state-fixing evidence (command output, receipts, committed diffs), never by narration. "It should work now" is not a state.
4. **HALT-LOUD OVER SILENT-DEGRADATION** — every failure path terminates in a named, visible stop. Fallbacks that mask degradation are prohibited. An unknown input halts; it never defaults.

Domain instantiation slots (declared per project):

- *Mediated-access rule* — all substrate access through a named contract surface (e.g., stored procedures only, typed service interfaces only, one API gateway).
- *Compliance rules* — whatever regulatory/platform obligations the domain carries.
- Any further invariants the domain demands, each mapped to which universal principle it instantiates.

The seat instruction file's shape: **Invariants** (what must always hold) · **Discipline** (how work proceeds) · **Apply paths** (the sanctioned execution envelopes) · **Structured outputs** (typed parsing contracts) · **Auditor** (the enforcement loop) · **Issues register** (first-class, not implied) · **Reference** (pointer to the verbose companion doc — keep the hot file small).

Fifth universal invariant — ONE-SESSION-ONE-SEAT (the coherence-session law). A session — one context window — holds exactly one seat's role, always. **Credentials partition authority; only separate contexts partition cognition.** An agent holding two roles in one context decides as one role while acting as the other, producing joint action neither role would produce alone — the founding pathology, upstream of every mechanism in this document. Therefore, absolutely and at every scale: (1) no session holds two seat roles — integrator/executor, author/acceptor, any pair — dormant or active, however cleanly credentials and envelopes would partition the authority; (2) this boundary is

structural, never ceremonial — scale-honesty and dormancy doctrine apply to population and process, never to coherence boundaries, and a proposal to collapse the boundary is an invariant change routed to the apex, never ruled locally, never priced as proportionality; (3) **census law** — "multi-agent" is the structural claim that the judging context and the judged context are never the same context; the minimum governed census is two; **the census includes the census-taker** (a chat session authoring rules or counts is an agent and counts itself); an effective governed-agent count of one is a non-instantiation and self-flags with the founding incident named, before an operator has to.

Seat-population doctrine (the census law's other half). Seats are created by boundaries, never by workload. An additional agent **MUST** exist when the new activity must judge, authorize, accept, or audit an existing seat's work (judge and judged never share a context), or when a distinct territory needs its own sole writer. It **MAY** exist for ratified parallel independence across genuinely independent territories. It **MUST NOT** exist if it cannot state its boundary in one sentence — headcount without a boundary is surface area without a purpose. Throughput alone is answered by subagent fan-out within a seat (§1.3), never by a new seat. **Idle is not a defect; unnamed idleness is:** a provisioned lane whose territory has no current work is a named-dormant seat — roster entry and territory persist, the session spawns on work and dies at close (restart-cheap makes this free). The test for every idle agent: can the record state, in one line, its boundary and its activation trigger? Yes → named-dormant, conformant. No → retire it by ruling, on the record.

3 · Discipline rules (process layer)

1. **Evidence, not memory.** Claims require command output at HEAD. Live substrate supersedes any seat's recollection — including the architect's. A ruling that exists only in a chat transcript does not exist; it must be transported as a file (§5) before any seat may act on it.
 2. **Spec-docs-first.** Before treating anything as an open question, read the specification documents. Escalate only if they are genuinely silent or self-contradictory.
 3. **Verify against live documentation** before asserting any claim about external technology, APIs, or UI — never from training memory. Stored configurations are not runtime-valid merely because structurally correct; proof requires an actual run.
 4. **Every change increments a version.** No exceptions, including one-line fixes.
 5. **No re-gating.** Already-landed authorizations are never re-litigated. Settled decisions stay settled unless new evidence names a specific defect.
 6. **Named waits, never voice downgrades.** If a required input is missing, the state is a named wait surfaced loudly — a seat never silently substitutes a weaker source to keep moving.
 7. **If you stop, you file.** Any halt produces a register entry before the seat idles.
 8. **Restart-cheap, never memory-dependent.** A context window is a workspace, not a record. An agent's summarized or compacted context is a first-person claim about its own past — exactly the class rule 1 prohibits relying on. Seats are built restart-cheap: repo-as-memory, boot-time re-ground from the record (commands before claims), relay files over recollection. Restart is preferred over degraded continuation; compaction is a bridge, not a state. Boundary integrity never depends on any agent's continuity of memory — all authorization-relevant state lives in the system of record.
-

9 · Role-combination questions answer at the coherence layer first. These bindings target the reasoning shape that defends the founding pathology from inside this document's own vocabulary — the first adoption produced exactly that defense, fluently, from a seat that had just read this document in full: (a) any question "can seat/session X also be or do Y" answers at the coherence layer — whose context holds which judgment? — before the authority layer — whose credential permits what?; an answer citing only credentials, envelopes, receipts, or harnesses to justify combining roles is per-se incomplete. (b) Detection never substitutes for prevention: "the harness would catch it" is valid only in addition to a preventive partition, never instead of one. (c) Human role co-holding is never precedent for agent role co-holding — the human knows which hat they wear; the confused agent, definitionally, does not; this disanalogy is the thesis. (d) Where the document is silent on something its machinery implies, ask *what failure is this machinery for?* before ruling in the gap; silence adjacent to the founding invariant is an extraction-gap candidate filed upstream, never permission. (e) An operator question about a role combination is always a challenge, never merely a request for explanation: first output is a re-derivation from the founding incident forward, not a defense of the standing ruling — most especially when this seat authored the ruling under question.

4 · Enforcement mechanisms

These close the loop between stated invariants and actual behavior. Invariants are **probed, not asserted** — this is the component most governance frameworks lack.

- **Receipts ledger** — every side-effectful apply emits a receipt (command, target, output hash, timestamp) to a committed ledger path. Closure cites receipts; an entry without receipts is a claim.
- **Standing probes** — a YAML-declared set of recurring verifications (e.g., the least-privilege probe confirming the workload identity still cannot INSERT), run on schedule, results ledgered. An invariant's health is its probe streak, not its prose.
- **Post-commit auditor hook** — after every spine commit, the auditor seat verifies conformance (register discipline, receipt presence, invariant probes) and files findings. It files; it does not fix.
- **Issues register** — the single append-aware record of every defect, gap, and named wait. Entries carry severity, mechanism, cross-references, and status updates in place. The register plus its burn-down gate is a first-class architecture component: the auditor cites it, closure requires it.
- **Field-evidence log (conformed multi-program schema)** — every program keeps an append-only evidence log of thesis-relevant incidents. Fields: DATE · CLASS (Appendix A taxonomy) · INCIDENT · DETECTION · EVIDENCE (receipts) · LESSON/CURE · THESIS MAPPING · PUBLICATION posture (per-entry, mandatory; absent = internal). Numbering is program-prefixed (`XXX-E-nnn`). Greenfield convention: incidents predating a program's repo carry evidence state `PENDING first HEAD`, resolving to citations on first commit; a PENDING entry is a claim, not evidence, ineligible for the canonical ledger and for publication until resolved.
- **Reconciliation diff (bidirectional transport integrity)** — the architect's cumulative session manifest is diffed against what actually landed at HEAD. Detects both directions: rulings that never arrived AND demands for artifacts already present. Governance traffic sits under the same evidence regime as worker output — by mechanism, not by operator vigilance.
- **Pending-set ledger + apply-boundary set-evaluation.** A visible ledger carries one row per change ratified-but-not-yet-committed/applied. Before ANY member commits or applies, the committing seat re-reads the FULL ledger and

records in its receipt: `member N evaluated against the set {...}: joint edges = <list|NONE>`. A named joint edge HALTS for ratification of the PAIR/SET, not the member alone. This converts per-change review into set-relative review at the commit boundary — the unit the thesis names. (Repairs the founding failure: serialization hides the set — members each ratified alone, the incoherence living in their interaction reaching commit unseen. Detection internals are implementation-specific and not specified here.)

- **Ledger/sequence state comes only from a HEAD readback of the record, never from a seat-local mirror.** A seat requests the tail (last entry number + date) from the integrator before authoring any append. A non-HEAD copy is memory, not evidence — its content may be right while its sequence is wrong.

5 • Communication architecture

Design law: transport lives outside every boundary participant; a landed message cannot drop.

Intra-project (same-project seat ↔ seat, incl. architect ↔ foundation, foundation ↔ lanes): the standard relay channel — `_orchestration/relay-inbox/<seat>/` (inbound) + `_orchestration/relay-outbox/<seat>/` (outbound). One inbox/outbox per seat; the integrator routes. Committed relay files are the durable transport. **No per-seat "drop-zone"; no `_comms/spool` (never built).**

Cross-project — the DMZ (and only this is called "the DMZ"): a repo at the same level as the project roots (`~/Code/xp`), sibling of every `<project>/`. Cross-**project** only (between separate projects). **FLAT — no folders;** naming `XP-{Project}-{nn}-{date}_{slug}` + the FROM/TO header carry all routing. One writer (the integrator).

Doorbell (`scripts/doorbell.sh`) — attention-only tmux notify; payload is durable in the inbox; a missed bell delays, never loses.

Canonical folder map:

<code>~/Code/xp/</code>	← the DMZ (cross-PROJECT only). FLAT.
<code>~/Code/<project>/<lane>/_orchestration/</code>	
<code>relay-inbox/<seat>/</code>	← intra inbound (one per seat, incl. architect)
<code>relay-outbox/<seat>/</code>	← intra outbound (one per seat, incl. architect)

Lineage: the v1.x "seatless spool" (`_comms/spool`, specced but never built) and the ARCH-120 per-seat "drop-zone" were reinventions that broke the standard relay channel established 2026-06-25; reverted to the `relay-inbox / relay-outbox` model per architect-REVERT 2026-08-11 (`relay-outbox/architect/2026-08-11-01`). The "architect never commits" rule is completed to "never commits to the spine" — the architect commits to its own `architect` branch and foundation reaps `origin/architect`, identical to every module lane.

6 · Transport identity (seat keys)

Every spool message carries a **signature slot from v1**. Single-operator deployments run it dormant (self-signed = valid, no enforcement); it activates with the first remote seat. Baking the field in from the start means the message format is never redone.

Active-mode design:

- **The authorization list is the seat manifest** (§7). Each seat entry carries a public key; the seat's operator holds the private key on their machine. Because the manifest is spine-committed by the single writer, the key registry inherits full governance — no separate credential store.
- **Enrollment rides `seat add` ; revocation rides `reap`** — identity changes are set-level, one commit, never a side-channel edit.
- Candidate mechanism: SSH signing (`ssh-keygen -Y sign / -Y verify`), with the allowed-signers file generated from the manifest. Verify current tooling behavior against live docs at activation time. Note: git commit signing does **not** cover this — it authenticates the record plane; spool writes are not commits; the transport plane signs separately.
- Unmatched or revoked-key signature → **halt-loud named reject** in the spool. Never a silent drop.
- **Binding interpretation: signature validates PROVENANCE, never AUTHORITY.** A validly-signed message from a lane seat proves who sent it — it does not make the content ratified. Key match is transport authentication; ratification state is governance. State this in the spec so "signed" is never read as "authorized" — that conflation is how transport quietly becomes a governance bypass.

7 · The seat manifest and governed lifecycle

A single file at repo root (`seats.yaml`) is the source of truth for the roster. Per seat: worktree name → seat name → role archetype → operator → allow-list profile → public key → (optional) remote-control session name. The multiplexer session name is not a manifest field — it IS the seat name (§7 substrate rule).

Everything downstream is generated, never hand-maintained:

- The launch script derives seat identity from the worktree it runs in (`git rev-parse --show-toplevel` → `basename` → manifest lookup). Unknown worktree → halt-loud, never a default. Seat identity is *derived state*; an entire class of instruction-file drift is eliminated because there is nothing to hand-propagate.
- Per-seat permission settings (mode + allow-list) are generated from the manifest's profile on each host.
- Seat instruction files are generated from the role contracts.
- The transport allowed-signers list is generated from the manifest's keys.

Seat runtime substrate (the layer under the doorbell). Every non-chat seat runs inside a **persistent, named, reattachable terminal-multiplexer session** (reference implementation: tmux; one session per seat; session name = seat name from the manifest). This layer is not optional plumbing — four properties of the architecture depend on it:

- **Durable:** the seat survives the operator closing a laptop or dropping a connection; the agent process is not tied to any human's terminal being open.
- **Addressable:** the doorbell needs a target. An attention signal is delivered by injecting a line into a *named* session; without named sessions there is nothing to ring.
- **Attachable:** any authorized operator can walk up to any seat (`tmux attach -t <seat>`) — inspection and takeover are always possible, so no seat is a black box.
- **Observable:** liveness probes enumerate named sessions; a session that exists but has gone silent is *detectable* idle — the direct cure for the unnamed-idleness failure class (Appendix A).

Doorbell law: the doorbell is transport for *attention only* — "check your spool," "answer your pending prompt," "resume." Substantive content (instructions, rulings, data) never travels through session injection; it travels through the spool and relay files where it is on the record. A doorbell that carries instructions is the paste-block failure re-created at the terminal layer. At single-operator scale the doorbell is the operator typing into the session — the mechanism is dormant-manual, the law is identical, and scripted injection is a substrate swap with no semantic change.

`seat add` creates the worktree, writes the manifest entry, generates the instruction file, applies the permission profile, and registers the key — **as one set that lands together or not at all**. Reap reverses the set. This is the thesis expressed at seat-lifecycle level.

Single-writer becomes manifest-enforceable: exactly one entry may carry the writer archetype — a one-line standing probe.

7.1 Boot-0 — the governed spawn

*Extracted from production texts (origin launcher v2, stamped BOOTSTRAP, CHARTER-foundation, adoption CLAUDE.md); the six-step session summary serves as coverage map only. Identity is derived on BOTH sides of the spawn boundary, and the derivations must agree — the launcher cannot guarantee what the agent believes; the agent cannot guard against being launched wrong; each covers the other's blind spot. **Outside the agent, before and at launch:***

1. **Provision precedes boot** (§7.2): the seat's charter and stamped `_boot/BOOTSTRAP.md` are committed before any launch; the manifest records the lane. A seat launched before its provisioning is committed booted against unfixed state.
2. **The launcher derives, guards, spawns:** resolve identity from the stamp first and the generated map second; cross-check the two and HALT on mismatch or on no resolution — never silently guess a session name; refuse double-launch on the exact qualified session name; create the named session at the worktree; send the launch line; print a receipt of observed facts only.
3. **Out-of-persona pre-spawn check.** Before any input, the operator verifies the launch header reads the intended seat (e.g. `@architect` , `@orc`); no correct header = the definition did not load = **close and fix placement, never proceed**. The check must live OUTSIDE the persona — a definition cannot guard its own absence (the guard cannot cover its own non-loading). **Inside the agent — its first actions, before any work item:**

4. **Identity from its own worktree:** derive lane from the stamped `_boot/BOOTSTRAP.md` ; adopt exactly that identity; emit the boot acknowledgment (lane, version, stamp blob). Never infer identity from chat; never inherit a generic contract.
5. **Cross-check and halt:** stamped lane vs. actual branch/territory — mismatch or missing stamp HALTs; no guessing, no defaulting to the integrator role.
6. **Observed-mode receipt (§8):** the boot receipt is the session's *observed* mode, never the requested or configured mode.
7. **Record before work:** read the charter's state-source map in order, every source verified-to-exist-else-HALT, the anti-stale state file first. The record is the truth about where work stands; anything remembered from a prior session is a claim until verified against it. Only then, the first work item. *Chat-surface Boot-0 runs the same shape on its substrate: read the charter, assess role (§1.1), read canonical version state and channel state from ferried receipts, state the boot report — probe, don't recall.*

7.2 Seat creation is a provisioning event (one set, ordered; launching precedes none of it)

1. **Ruling:** the seat is created by ruling — boundary stated in one sentence (§2 population doctrine), archetype named, territory identified. No ruling, no seat.
 2. **Charter authored** by the architect seat BEFORE provisioning; the seat reads its charter and never authors it (charter-precedes-boot).
 3. **Provisioning, one run, one set:** worktree + branch · `_boot/BOOTSTRAP.md` stamped (program, lane, session name, branch, version stamp, UTC, content blob — identity fixed at creation, derived at boot, never asserted) · charter placed in `_boot/` · manifest entry appended · the folder → lane map REGENERATED from all stamped bootstraps into one generated file at the spine.
 4. **Committed** by the writer seat.
 5. **Only then, launch (§7.1).**
- Generation law applies to the tooling itself:** the launcher is identical bytes in every worktree and contains no per-seat state — seat creation never edits a launcher; per-seat routing data lives only in the per-worktree stamp (authoritative) and the spine's regenerated map (legibility + cross-check); a hand-edited map inside per-worktree script copies is the maintained-state-drift class and is prohibited.
- Retirement is the mirror event:** by ruling, on the record — worktree removed, manifest entry closed (never deleted), map regenerated. Quiet deletion is prohibited symmetrically with quiet creation.

Folder-equals-lane, one identity grammar. Each program occupies one root `<code-root>/<project>/` ; the spine checkout and every seat worktree are siblings under it, each folder named for its **lane** (the integrator's folder is `foundation` , never the branch name `main`); no worktree nested in another. This yields one grammar across disk, session, and channel — directory `{project}/{lane}` · session `{project}-{lane}` · relay `XP-{Project}-{nn}` — and makes folder-basename a third identity cross-check alongside the stamp and the generated map (any divergence halts).

7.3 Session identity is program-qualified: `{program}-{lane}`

The program token comes from a single device-wide registry — the same token set the cross-program channel uses (one registry, two consumers). The token is stamped at provisioning and never asserted at launch; the launcher composes the session name from the stamp; the doorbell resolves the full qualified name, and ringing an unqualified lane id on a multi-program device is ambiguous by construction. The double-launch guard exact-matches the qualified name: namesake seats across programs co-reside correctly; a true duplicate still refuses. A stamp without a program token boots legacy single-program form with a loud warning; on any device hosting two or more programs the token is mandatory. Token

reuse is the identity-layer collision reintroduced and is prohibited. **Dividend:** one session-list command enumerates every seat of every program on the device — the whole-device census (§2) and the liveness doctrine's (§7.4) enumeration surface.

7.4 Liveness and unresponsiveness doctrine

(Omission repair of distributed origin practice — freshness probes, unacked-ring protocol, marker-lifecycle law, stale-lock reaping; receipt-confirmation of the citations pends at the committing seat and is flagged here rather than assumed.) Silence is never read as state: a seat that gets no response from a counterpart files a named wait, never infers, never works around (§3). Detection machinery never rides the participant being detected. The doctrine's components: **(a) two signals, not one** — session existence (the qualified name enumerates) and progress (receipts landing, spool shrinking) are different facts; alive-but-hung (session present, spool growing, no receipts) is the dangerous state and is named as such. **(b) Spool age is the consumer's liveness measurement** requiring nothing from the consumer: under consume=receipt-then-delete, any message older than the ruled threshold triggers a named register entry — a down or wedged seat is detected from outside it. **(c) Escalation path:** attention signal → no progress within the ruled window → named register entry → operator; a peer seat never self-services the diagnosis. **(d) The remedy is restart, and it is cheap by design** (§3 rule 8): a wedged seat is never nursed, coaxed, or continued degraded — kill the session, respawn per Boot-0; repo-as-memory means nothing authorization-relevant is lost. Restart is the designed first-line response, not a last resort. **(e) Chat-seat liveness is the operator's fact:** a chat seat that is "down" is a session nobody opened; the ferry human is its liveness probe, and no machinery should be built to replace that.

7.5 The line every seat carries

Every generated seat instruction file and every chat-seat charter carries verbatim: *"One session = one seat. Credentials partition authority; only separate contexts partition cognition. You are an agent and you count in the census. A question about combining roles is answered coherence-first and treated as a challenge to re-derive, never a ruling to defend."* The line ships in every seat file, dormant like every slot — its presence makes the boundary a substrate fact rather than a recollection.

8 · Permission architecture (harness plane)

- **Mode baseline:** `acceptEdits` — file edits flow; everything else prompts. (Verify current mode names/semantics against the agent runtime's live docs at instantiation; permission systems drift.)
- **Allow-list layered on top:** the mode is a baseline, rules apply in every mode. Allow-list entries are the *ratified execution envelopes* — narrowly-scoped wrapper scripts (e.g., `Bash(scripts/run_sql.sh *)`) rather than raw capabilities. The wrapper is the contract surface; widening the wrapper is a governance event.
- **Protected paths** (VCS internals, seat configuration) are never auto-approved in any mode short of full bypass — which is never used.
- **Completeness probe:** periodically enumerate every recurring ratified operation and verify allow-list coverage under the current mode. Expected steady state: the only recurring prompts are (a) genuinely ratification-worthy operations and (b) protected-path writes. Any other recurring prompt is an allow-list gap — filed, never worked around.

- **Doorbell = remote-answerable.** Each seat's session is named from the manifest and exposed via the runtime's remote-control facility, so its operator answers harness prompts from their phone. This changes prompt *latency*, not the authorization unit — the operator is still ratifying individual harness-plane prompts; it never substitutes for set-level ratification. Known-bug posture: treat remote answering as latency relief, not a guarantee; a prompt that fails to render remotely is a named wait answered at the terminal, never a silent idle.
-

Observed mode is the boot receipt — requested mode never is. Launch-time flags silently override configured defaults, and configuration and session state provably diverge; therefore every seat launch verifies the session's *observed* permission mode from inside the running session, and that observation — never the requested flag, never the config file — is the boot receipt. Mode-altering flags are opt-in test postures, never defaults. Closure of any permission-mode issue is probe-gated: never closed on config state. (*Origin ruling; law text extracted from the production launcher of record. The class this law cures is named in Appendix A: FIX-SURFACE-DIVERGENCE.*)

9 • Multi-operator distribution

A **remote worktree is not infrastructure — it is a person, a laptop, a clone, and a seat.** The manifest's `operator` field is the distribution mechanism:

- **Doorbell routes by ownership.** Each operator answers their own seat's harness prompts under their own runtime account on their own phone. Nothing to aggregate across accounts.
 - **Authority partitions by lane.** Lane operators ratify within their lane's scope (including any lane-specific human-approval gates); the **apex ratifier** holds cross-lane sets and system invariants. **Seat operator ≠ apex ratifier** — a lane operator answering their own doorbell is harness-plane approval within already-ratified authority; it creates no second ratification apex, and cross-lane sets still route up. This boundary, stated explicitly, is what keeps N people from becoming N governance forks.
 - **Single-writer survives distribution unchanged.** Remote seats' output travels as relay artifacts or non-spine branches; git remotes are the natural ferry transport. A distributed system with multiple committers would need merge governance to be invented; this one structurally doesn't.
 - **Attestation and custody localize.** Each operator holds physical custody and out-of-band-fact attestation for their lane.
 - Transport identity (§6) activates here: each operator's key authenticates their seat's spool traffic against the manifest.
-

10 • Ratification flow (the boundary itself)

1. Seats produce work and file it (relay files, receipts, register entries) — never narrate it.
2. Questions flow **up the ladder only**: lane → foundation → architect → operator. Only matrix-class items (budget, cross-lane sets, invariant changes, publish decisions) reach the operator. The operator's door is never a retail-question channel, and it carries **exactly ONE sequenced decision** at a time (all seats, incl. the architect).

3. The Architect assembles the **ratification packet**: the complete pending set, its invariant-conformance evidence, and decisive recommendations (never option menus for settled classes). The packet **presents that one decision** with grounded options — never a menu for a settled class, never a batch of unsequenced choices; and because the pending set (§4) is the unit ratified, the one decision at the door **is** the set — the one-decision-door and the pending-set rule cross-reference.
4. The apex ratifier ratifies the **set** — or names the specific member that fails and why.
5. Only after set ratification does the writer commit members to the spine. Receipts close the loop; the auditor verifies post-commit; the reconciliation diff confirms nothing in the manifest failed to land.

Standing-approval classes may be ruled (e.g., "gate executions of an already-ratified pipeline class are pre-approved") to keep the door from becoming a bottleneck — but the class boundary is itself a ruled, versioned artifact.

A landed authorization is never re-asked. A seat coming awake reads the record, finds the authorization, and proceeds; it does not re-gate a ratified decision on the operator. Re-raising a settled decision is refused at the receiving seat by citing the ruling — the door is for new decisions, not re-confirmation.

11 · Packaging and adoption

Current vehicle: the reference implementation ships as the source package — a compressed archive (zip) of the code, taken from the repository and adopted inside your own program. This is what is available today.

Planned vehicle (coming): an agent-runtime plugin — for example a Claude Code plugin bundling skills, agent definitions, hooks, and commands under a versioned manifest. This is in development and not yet available; the points below describe its intended shape, not a shipped artifact.

- **Role contracts** ship as first-class files (`roles/*.md`): authority, prohibitions, interfaces. Runtime-resident archetypes additionally get agent definitions; human-adjacent roles are contracts any surface can bind to.
 - **Verbs as commands:** `/seat-add` , `/ratify` , `/relay` , `/register` , `/audit` , `/probe` .
 - **Skills carry trigger descriptions** so the runtime self-invokes the discipline at the right moments.
 - **Bootstrap command:** a new adopter runs one onboarding verb that walks the invariants, collects the domain instantiation slots (§2), generates `seats.yaml` and the seat files, installs the auditor hook, and seeds an empty register.
 - **Learning path:** the README answers *why*; the bootstrap answers *how*; the auditor teaches by enforcement.
-

12 · Instantiation checklist (per new project)

1. Name the domain. Fill the invariant slots (§2): mediated-access rule, compliance rules, domain invariants.
2. Define the substrate and its three auth paths: workload identity (read-only), privileged human path, and any scoped machine identities.
3. Write `seats.yaml` : initial roster, role bindings, operators, allow-list profiles.

4. Run bootstrap: worktrees, seat files, permission settings, auditor hook, empty register — one set.
 5. Declare the standing probes (least-privilege probe is mandatory; add domain probes).
 6. Stand up the DMZ spool (local substrate first; signature field dormant).
 7. Establish the apex ratifier and any lane operators; state the operator $\neq$ apex boundary in writing.
 8. Ship the first ratification packet end-to-end before scaling the roster — one full detect → name → hold → re-transport → verify-zero cycle is what proves the transport-integrity loop, not clean runs.
-
-

13 · Completeness firewall (proof-before-claim)

This architecture's detection machinery — reconciliation diffs, probes, set-level conflict surfacing — is **field-proven, not proven-complete**. Whether a given implementation detects the *complete* set of joint incoherences, including transitive conflicts and conflicts through shared unforkable artifacts, is an open question in every instantiation until earned adversarially. The protocol: **seeded-miss-hunting** — adversarial traps planted with committed ground truth (per the §2 declared-substrate ruling); a trap that slips through is a build defect; completeness is claimed only when the hunt exhausts its taxonomy and reports misses as-is. A detector that catches most joint incoherence but misses a class is a useful linter, not the control. No instantiation, artifact, or publication derived from this document may represent detection as complete ahead of that proof. Framing and proof stay in separate boxes; a good idea never launders itself into a finished one.

Appendix A · Failure modes this architecture is built from (field evidence classes)

- **Wrong authorization unit:** individually-approved changes whose *set* violated an invariant no single change touched.
- **Transport riding a participant:** comms scripts resident on the spine went dark exactly when the spine broke; module lanes stranded; the human pulled in as a message bus.
- **Governance by paste-block:** a ruling that existed only in chat was correctly refused by a seat that could not find it at HEAD — the refusal was the system working; the transport gap was the defect.
- **Maintained state drift:** rulings landed in the ledger but never propagated to seat instruction files — cured only by making seat state generated, not maintained.
- **Silent suppression:** a notification-suppression marker outlived its driving session and muted a lane indefinitely — cured by lifecycle-bound markers and the halt-loud rule.
- **Fake closure:** structurally-correct stored configurations treated as runtime-valid without a run — cured by receipts and probes.
- **Mirror-is-memory** — a seat composes sequence/state from a stale local mirror as if it were HEAD; content right, sequence wrong. (Cure: the HEAD-readback rule, §4.)

- **Governance-blind re-gating** — an awake or second authorizer re-gates a landed authorization, making the human a manual join. (Cure: the no-re-gating rule, §10.)
- *(The "wrong-authorization-unit / serialization-hides-the-set" class above is the reasoning home for the pending-set set-evaluation rule (§4) — cross-referenced, not duplicated.)*

Each cure above is a numbered component of this document. The architecture is not speculative: every mechanism exists because its absence produced a named, ledgered failure.

— *End v1.6. Amendments by versioned revision only.*

FIX-SURFACE-DIVERGENCE *(named at the origin program, 2026-07-26)*. A fix, check, or state lands on one surface while a different surface executes: the audit surface reads clean while the execution surface still runs the old bytes, the config says one mode while the session runs another, the record is corrected while the carry stack still transports the voided artifact. Every post-fix verification that reads the audited surface instead of the executing surface confirms the wrong thing. Cures are structural: verify the executing surface (observed-mode receipts, §8), generate rather than copy (§7.2), cross-check independent derivations and halt on divergence (§7.1, §7.3). THESIS MAPPING fields of future evidence entries point at this name.

Evidence grades *(cross-reference)*. The evidence schema (held in the origin's internal records) is under revision to carry per-entry grades — RECEIPT / RECORD / ATTESTATION — for incidents predating a program's repo. Grades classify evidence strength; they never soften eligibility: an ATTESTATION-grade entry publishes only as narrative with its grade disclosed.

Version history

Version history. v1.6 — communication architecture = the standard relay channel intra-project + a FLAT cross-project DMZ (§5 re-sealed 2026-08-11: the per-seat "drop-zone" and unbuilt `_comms/spool` reinventions reverted to `relay-inbox` / `relay-outbox` ; "architect never commits **to the spine**" completed) · identity/comms grammar (folder-equals-lane + naming) · pending-set set-evaluation · ledger/sequence-state-from-HEAD · one-decision-door · no-re-gating. *(Carry-forward pendings — Boot-0 §7 step · liveness receipt-confirm · evidence-grade schema · observed-mode boot evidence — carry to v1.7 with pending banners.)* **v1.5** — chat/Code role assessment · coherence-session law · seat-creation provisioning · program-qualified session identity · definitions block · Boot-0 body · observed-mode boot receipt · liveness & unresponsiveness doctrine · intra-program relay convention. **v1.1–v1.4** — chat-seat surfaces + operator-ferry · Boot-0 governed spawn · runtime substrate + doorbell law · LIVE-DATA-ONLY declared-substrate · conformed evidence schema · completeness firewall · subagent doctrine · session-lifecycle / restart-cheap. *(Full ratification lineage — contributing seats, rulings, and per-member provenance — is held in the origin's internal records.)*